

# SheetEX v2 — Teacher Guide

Data wrangling for AI across Excel 365, Excel 2013, Google Sheets, CSV/TSV and SQL.

9 lessons · 81 auto-checked challenges · about 11 class periods (50 min) · certificates for every lesson

Teacher-only: this guide contains the answer key. It is NOT part of the student app.

## How it runs

- **Students** open your Apps Script web-app link (embedded in Canvas). Progress saves in their browser automatically.
- **Certificates** unlock when every challenge in a lesson is done. Students print/save as PDF, download an image, or paste the certificate code into Canvas.
- **Gradebook** (Apps Script only): every certificate is recorded in the *SheetEX Gradebook* Google Sheet in your Drive. Run `setup` once in the script editor to get its link.
- **Checking codes:** in the app click **? ► For teachers** and paste all submitted codes at once.
- **Hints** cost 20% of a challenge's XP; a certificate shows hints used, so you can see independence at a glance.
- **Moving computers:** students use *Class cloud save* (name + PIN) or copy their progress code.

## Suggested pacing

| # | Lesson                                                                  | Challenges | XP  | Time   |
|---|-------------------------------------------------------------------------|------------|-----|--------|
| 1 | <b>Spreadsheet Foundations</b><br>Excel 365                             | 10         | 225 | 50 min |
| 2 | <b>Classic Excel &amp; Nested Functions</b><br>Excel 2013               | 8          | 190 | 50 min |
| 3 | <b>Google Sheets Survival Guide</b><br>Google Sheets                    | 9          | 205 | 50 min |
| 4 | <b>Data Files: CSV &amp; TSV</b><br>CSV / TSV                           | 8          | 135 | 45 min |
| 5 | <b>Data Wrangling I: Cleaning Data for AI</b><br>Data Wrangling Lab     | 11         | 220 | 90 min |
| 6 | <b>Data Wrangling II: Combining &amp; Lookups</b><br>Data Wrangling Lab | 11         | 220 | 90 min |
| 7 | <b>SQL for Data Analysts</b><br>SQL Database                            | 9          | 215 | 75 min |

|   |                                                           |    |     |        |
|---|-----------------------------------------------------------|----|-----|--------|
| 8 | <b>Cross-Platform Translator</b><br>Compare               | 5  | 60  | 25 min |
| 9 | <b>Databases 101: What is an RDBMS?</b><br>RDBMS Explorer | 10 | 175 | 60 min |

Lessons 1–4 can be done in any order. Lessons 5–6 (Data Wrangling) are the core of the AI-data unit — the 🤖 AI check tab shows students how their cleaning changes what a model learns. Lesson 8 makes a good review day.

Differentiation: early finishers can switch the Data Wrangling Lab engine (Excel 365 ⇌ Google Sheets) and redo it in the other app, or explore the Interactive Cheat Sheet's SQL column.

# Lesson 1: Spreadsheet Foundations

Workspace: **Excel 365** · 10 challenges · about 50 minutes

Spreadsheets are where most data work starts — including the data used to train and test AI. Modern Excel can answer questions about thousands of rows with one formula.

## Students will be able to

- SUM, COUNTIF, SUMIF
- XLOOKUP
- FILTER, SORT, UNIQUE (dynamic arrays)
- IFS and LET
- Absolute vs relative references

## Common misconceptions

- #SPILL! means something is in the way of a spilling result — clear the cells below.
- Fill formulas down with Ctrl+D; check the \$ signs before you do.
- A typed number is frozen; a formula updates when the data changes.

## Vocabulary

|                                       |                                                                  |                                                  |
|---------------------------------------|------------------------------------------------------------------|--------------------------------------------------|
| <b>Formula</b>                        | Starts with = and calculates something.                          | <code>=SUM(F2:F25)</code>                        |
| <b>Range</b>                          | A rectangle of cells, written first:last.                        | <code>Products!D2:D25</code>                     |
| <b>Relative vs absolute reference</b> | A2 shifts when copied; \$A\$2 stays put.                         | <code>=E2*\$H\$1</code>                          |
| <b>Function argument</b>              | The values inside the parentheses, separated by commas.          | <code>SUMIF(range, criteria, sum_range)</code>   |
| <b>Dynamic array / spill</b>          | One formula returns many results that flow into the cells below. | <code>=FILTER(B2:B25, C2:C25="Drinks")</code>    |
| <b>Lookup</b>                         | Find a row by a key and return another column from it.           | <code>=XLOOKUP("SKU-404", A2:A25, B2:B25)</code> |

## Challenges & answer key

| # | Challenge                                | Task                                                                                                                                                                                   | Answer                                           |
|---|------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------|
| 1 | <b>Your First Formula</b><br>★☆☆ · 10 XP | In Scratch!B2, find the total number of items in stock (the InStock column on the Products sheet).<br><br><i>Takeaway: SheetName!Range lets one sheet use data from another sheet.</i> | Scratch!B2<br><code>=SUM(Products!F2:F25)</code> |

|   |                                             |                                                                                                                                                                                                                                                                                        |                                                                                                                                                  |
|---|---------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| 2 | <b>What Is It All Worth?</b><br>★★★ · 15 XP | <p>In Scratch!B3, calculate the total value of the inventory: every product's Price × InStock, all added up — in ONE formula.</p> <p>Takeaway: SUMPRODUCT works on every version. SUM(D2:D25*F2:F25) only works in Excel 365 (or with Ctrl+Shift+Enter in old Excel).</p>              | <p>Scratch!B3</p> <pre>=SUMPRODUCT(Products!D2:D25,Products!F2:F25)</pre> <p>Also accepted:</p> <pre>=SUM(Products!D2:D25*Products!F2:F25)</pre> |
| 3 | <b>Count the Categories</b><br>★★★ · 15 XP  | <p>In Scratch!B4, count how many products are in the School Supplies category.</p> <p>Takeaway: COUNTIF and SUMIF work the same in every spreadsheet app.</p>                                                                                                                          | <p>Scratch!B4</p> <pre>=COUNTIF(Products!C2:C25,"School Supplies")</pre>                                                                         |
| 4 | <b>Store S03 Sales</b><br>★★★ · 20 XP       | <p>In Scratch!B5, add up the Qty of every sale made by store S03 (Sales sheet).</p> <p>Takeaway: SUMIF = "add these numbers, but only on rows where something is true." In SQL it is SUM(...) WHERE ...</p>                                                                            | <p>Scratch!B5</p> <pre>=SUMIF(Sales!C2:C61,"S03",Sales!E2:E61)</pre>                                                                             |
| 5 | <b>XLOOKUP Detective</b><br>★★★ · 20 XP     | <p>Type SKU-404 into Scratch!A7. In Scratch!B7, use XLOOKUP to show that SKU's product name. It should still work if someone types a different SKU into A7!</p> <p>Takeaway: XLOOKUP only exists in Excel 365/2021 and Google Sheets. Excel 2013 shows #NAME? — try it there next!</p> | <p>Scratch!A7</p> <pre>SKU-404</pre> <p>Scratch!B7</p> <pre>=XLOOKUP(A7,Products!A2:A25,Products!B2:B25)</pre>                                   |
| 6 | <b>Spill the Drinks</b><br>★★★ · 25 XP      | <p>In Scratch!D2, write ONE formula that lists the names of every product in the Drinks category. The answers should spill down.</p> <p>Takeaway: If anything is typed in the cells below D2, Excel shows #SPILL! because the answer has no room.</p>                                  | <p>Scratch!D2</p> <pre>=FILTER(Products!B2:B25,Products!C2:C25="Drinks")</pre>                                                                   |
| 7 | <b>Sorted &amp; Unique</b><br>★★★ · 25 XP   | <p>In Scratch!F2, list each product category once, sorted A→Z, in one formula.</p> <p>Takeaway: Nesting = putting a function inside another function. The inside one runs first.</p>                                                                                                   | <p>Scratch!F2</p> <pre>=SORT(UNIQUE(Products!C2:C25))</pre>                                                                                      |

|                                                 |                                                                                                                                                                                                                                                     |                                                                                                                                                   |
|-------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>8 Stock Status with IFS</b><br>★★★ · 30 XP   | On the Products sheet, type Status in I1. In I2:I25 show "Reorder" if InStock is below ReorderAt, "Low" if InStock is below ReorderAt × 1.5, otherwise "OK". Use IFS.<br>Takeaway: IFS is new. In Excel 2013 you would need IF inside IF inside IF. | Products!I1<br><b>Status</b><br>Products!I2:I25 (type in the first cell, fill down)<br>=IFS(F2<G2,"Reorder",F2<G2*1.5,"Low",TRUE,"OK")            |
| <b>9 LET It Be</b><br>★★★ · 30 XP               | In Scratch!B9, calculate the profit margin of SKU-502: (price – cost) ÷ price. Use LET to name the price and the cost.<br>Takeaway: LET makes long formulas readable — and Excel only calculates each name once.                                    | Scratch!B9<br>=LET(p,XLOOKUP("SKU-502",Products!A2:A25,Products!D2:D25),c,XLOOKUP("SKU-502",Products!A2:A25,Products!E2:E25),(p-c)/p)             |
| <b>10 Build a Revenue Column</b><br>★★★ · 35 XP | On the Sales sheet, type Revenue in G1. Fill G2:G61 with Qty × Price. The price is on the Products sheet, so you will need a lookup.<br>Takeaway: \$A\$2 is an ABSOLUTE reference: it does not change when copied. A2 is RELATIVE: it shifts.       | Sales!G1<br><b>Revenue</b><br>Sales!G2:G61 (type in the first cell, fill down)<br>=E2*XLOOKUP(D2,Products!\$A\$2:\$A\$25,Products!\$D\$2:\$D\$25) |

## Discussion

1. Why is a formula better than typing the answer, even if the answer is right today?
2. When would you use \$A\$2 instead of A2?

## Exit ticket

Write a formula that counts how many products cost more than \$10.

Answer: =COUNTIF(Products!D2:D25,">10")

## Lesson 2: Classic Excel & Nested Functions

Workspace: **Excel 2013** · 8 challenges · about 50 minutes

Not everyone has the newest software. Building answers from classic functions teaches you how lookups and logic really work — and your files will open on any computer.

### Students will be able to

- INDEX + MATCH
- VLOOKUP exact match
- Nested IF
- IFERROR
- Ctrl+Shift+Enter array formulas

### Common misconceptions

- Forgetting FALSE in VLOOKUP gives wrong answers with no error.
- Without Ctrl+Shift+Enter, array formulas quietly use only one row.
- Count your parentheses: every IF( needs a ).

### Vocabulary

|                                   |                                                                          |                                                                   |
|-----------------------------------|--------------------------------------------------------------------------|-------------------------------------------------------------------|
| <b>Nesting</b>                    | Putting one function inside another. The inside one runs first.          | <code>=IF(A2&gt;10, "Big", IF(A2&gt;5, "Medium", "Small"))</code> |
| <b>INDEX / MATCH</b>              | MATCH finds the position; INDEX returns the value at that position.      | <code>=INDEX(B2:B25, MATCH("SKU-404", A2:A25, 0))</code>          |
| <b>Exact vs approximate match</b> | VLOOKUP needs FALSE for exact matches.                                   | <code>=VLOOKUP(x, A2:D25, 3, FALSE)</code>                        |
| <b>Array formula (CSE)</b>        | A formula that works on whole ranges, confirmed with Ctrl+Shift+Enter.   | <code>{=MAX(IF(C2:C25="Snacks", D2:D25))}</code>                  |
| <b>#NAME?</b>                     | The program does not know a function name — often because it is too new. | <code>=XL00KUP(...)</code> in Excel 2013                          |

### Challenges & answer key

| # | Challenge                          | Task                                                                                                                                                                                                                          | Answer                                                                                                                                         |
|---|------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | <b>Back to 2013</b><br>☆☆☆ · 10 XP | In Scratch!B2, try a modern formula:<br><code>=XLOOKUP("SKU-101", Products!A2:A25, Products!B2:B25)</code> .<br>What does Excel 2013 do?<br><br>Takeaway: #NAME? means "I don't know that word." Old Excel has never heard of | Scratch!B2<br><code>=XL00KUP("SKU-101", Products!A2:A25, Products!B2:B25)</code><br><br>The correct result is #NAME? — the point is to see it. |

## XLOOKUP.

|   |                                                  |                                                                                                                                                                                                                                                                                                        |                                                                                                                                                                                       |
|---|--------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 2 | <b>INDEX + MATCH</b><br>★★★ · 25 XP              | In Scratch!B3, show the product name for SKU-404 using INDEX and MATCH nested together (no VLOOKUP).<br><br>Takeaway: INDEX/MATCH works in EVERY version of Excel and Google Sheets. It is the universal lookup.                                                                                       | Scratch!B3<br><code>=INDEX(Products!B2:B25,MATCH("SKU-404",Products!A2:A25,0))</code>                                                                                                 |
| 3 | <b>VLOOKUP's Trap</b><br>★★★ · 20 XP             | In Scratch!B4, use VLOOKUP to find the price of the Hoodie by its name. Make sure you get the RIGHT price!<br><br>Takeaway: Always finish VLOOKUP with FALSE (exact match). Forgetting it is the #1 VLOOKUP bug in the world.                                                                          | Scratch!B4<br><code>=VLOOKUP("Hoodie",Products!B2:D25,3,FALSE)</code><br><br>Without FALSE the answer is \$5.99 (wrong) — a great discussion point.                                   |
| 4 | <b>The Nested IF Ladder</b><br>★★★ · 30 XP       | On Products, fill I2:I25 with "Reorder" (InStock below ReorderAt), "Low" (below ReorderAt × 1.5) or "OK". IFS doesn't exist here — nest IF inside IF.<br><br>Takeaway: Every IFS can be rewritten as nested IFs. Count your parentheses — each IF( needs a ).                                          | Products!I2:I25 (type in the first cell, fill down)<br><code>=IF(F2&lt;G2,"Reorder",IF(F2&lt;G2*1.5,"Low","OK"))</code>                                                               |
| 5 | <b>IFERROR Safety Net</b><br>★★★ · 20 XP         | Type any SKU in Scratch!A6. In Scratch!B6, show its product name — or the words Not found if the SKU doesn't exist.<br><br>Takeaway: IFERROR catches any error. IFNA (Excel 2013+) only catches #N/A, which is safer for lookups.                                                                      | Scratch!A6<br>SKU-101<br><br>Scratch!B6<br><code>=IFERROR(VLOOKUP(A6,Products!A2:B25,2,FALSE),"Not found")</code>                                                                     |
| 6 | <b>The Ctrl+Shift+Enter Club</b><br>★★★★ · 40 XP | In Scratch!B7, find the highest Price in the Electronics category. MAXIFS does not exist in 2013! Use MAX(IF(...)) and confirm with Ctrl+Shift+Enter.<br><br>Takeaway: Before dynamic arrays, formulas that work on whole ranges needed Ctrl+Shift+Enter. Without it, Excel quietly used only ONE row. | Scratch!B7<br><code>=MAX(IF(Products!C2:C25="Electronics",Products!D2:D25))</code><br><i>Ctrl+Shift+Enter</i><br><br>Confirm with Ctrl+Shift+Enter. Plain Enter gives a wrong answer. |
| 7 | <b>Build a Price Tag</b><br>★★★ · 15 XP          | In Scratch!B8, build the text Granola Bar - \$1.50 from Products!B2 and Products!D2.<br><br>Takeaway: Formatting (\$) is not part of the number. TEXT(...) bakes the format into text.                                                                                                                 | Scratch!B8<br><code>=Products!B2&amp;" - "&amp;TEXT(Products!D2,"\$0.00")</code>                                                                                                      |
| 8 | <b>Wildcard SUMIFS</b><br>★★★★ · 30 XP           | In Scratch!B9, add up the Qty of sales from store S02 for products whose SKU starts with SKU-3 (school supplies).                                                                                                                                                                                      | Scratch!B9<br><code>=SUMIFS(Sales!E2:E61,Sales!C2:C61,"S02",Sales!D2:D61,"SKU-3*")</code>                                                                                             |

Takeaway: SUMIFS/COUNTIFS have existed since Excel 2007, so they are safe to use everywhere.

---

## Discussion

1. Your formula works at school but shows #NAME? at home. What are two possible reasons?
2. Why did people love SUMPRODUCT before dynamic arrays existed?

## Exit ticket

Rewrite =XLOOKUP("SKU-201", A2:A25, B2:B25) so it works in Excel 2013.

Answer: `=INDEX(B2:B25, MATCH("SKU-201", A2:A25, 0))`

# Lesson 3: Google Sheets Survival Guide

Workspace: **Google Sheets** · 9 challenges · about 50 minutes

Google Sheets runs on every Chromebook and is built for sharing. It looks like Excel, but its functions do not always take the same arguments — the #1 reason "it worked at school but not at home".

## Students will be able to

- FILTER and SORT the Sheets way
- ARRAYFORMULA
- QUERY
- SPLIT and REGEXMATCH
- Open-ended ranges

## Common misconceptions

- SORT(..., -1) sorts ASCENDING in Sheets — no error, wrong answer.
- CONCAT only joins two values in Sheets.
- A bad formula is saved and shows #ERROR! instead of a pop-up.

## Vocabulary

|                          |                                                                               |                                                            |
|--------------------------|-------------------------------------------------------------------------------|------------------------------------------------------------|
| <b>ARRAYFORMULA</b>      | Lets normal formulas work on whole ranges and fill a column with one formula. | <code>=ARRAYFORMULA(D2:D25-E2:E25)</code>                  |
| <b>FILTER conditions</b> | Each condition is its own argument (AND).                                     | <code>=FILTER(B2:B25, C2:C25="Snacks", D2:D25&gt;1)</code> |
| <b>is_ascending</b>      | SORT direction is TRUE/FALSE in Sheets.                                       | <code>=SORT(B2:D25, 3, FALSE)</code>                       |
| <b>QUERY</b>             | SQL-like questions inside a spreadsheet; columns are letters.                 | <code>=QUERY(A1:H25, "select C, sum(F) group by C")</code> |
| <b>Open-ended range</b>  | E2:E means "E2 to the bottom".                                                | <code>=SUM(Sales!E2:E)</code>                              |

## Challenges & answer key

| # | Challenge                                        | Task                                                                                                                                                                                 | Answer                                                                                      |
|---|--------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| 1 | <b>Excel Habits, Sheets Rules</b><br>★☆☆ · 20 XP | In Scratch!B2, type the Excel-style formula <code>=FILTER(Products!B2:B25, Products!C2:C25="Apparel", "None")</code> . Read the error, then fix it so it lists the Apparel products. | <code>Scratch!B2</code><br><code>=FILTER(Products!B2:B25, Products!C2:C25="Apparel")</code> |

Takeaway: Same function name, different arguments. In Sheets every extra FILTER argument is another condition.

|   |                                                       |                                                                                                                                                                                                                                                                            |                                                                                                   |
|---|-------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| 2 | <b>Two Conditions</b><br>★★☆ · 25 XP                  | In Scratch!D2, list Snacks that cost more than \$1.00. Give FILTER the two conditions as two separate arguments (the Sheets way).<br><br>Takeaway: Excel needs one combined test instead: (cond1)*(cond2). The translator panel shows both!                                | Scratch!D2<br><code>=FILTER(Products!B2:B25,Products!C2:C25="Snacks",Products!D2:D25&gt;1)</code> |
| 3 | <b>Which Way Is Down?</b><br>★★☆ · 20 XP              | In Scratch!F2, sort Products!B2:D25 by price, highest first. (Hint: Excel's -1 will NOT work here — try it and see!)<br><br>Takeaway: In Sheets, -1 counts as TRUE (any non-zero number is TRUE) so it sorted the WRONG way with no error. Silent bugs are the worst bugs. | Scratch!F2<br><code>=SORT(Products!B2:D25,3,FALSE)</code>                                         |
| 4 | <b>One Formula, 24 Answers</b><br>★★☆ · 30 XP         | On Products, put ONE formula in I2 that fills I2:I25 with the profit per unit (Price – Cost). Use ARRAYFORMULA.<br><br>Takeaway: In Excel 365 this just works without ARRAYFORMULA. In Excel 2013 it needs Ctrl+Shift+Enter across the whole range.                        | Products!I2<br><code>=ARRAYFORMULA(D2:D25-E2:E25)</code>                                          |
| 5 | <b>QUERY: SQL Inside a Spreadsheet</b><br>★★★ · 35 XP | In Scratch!J2, use QUERY to show each category and its total InStock.<br><br>Takeaway: QUERY is Google-only. It is basically SQL — head over to SQL mode to see the real thing.                                                                                            | Scratch!J2<br><code>=QUERY(Products!A1:H25,"select C, sum(F) group by C")</code>                  |
| 6 | <b>SPLIT the Manager</b><br>★★☆ · 15 XP               | In Scratch!B10, split the manager name in Stores!E2 into first name (B10) and last name (C10).<br><br>Takeaway: SPLIT is Google-only. Excel uses Data ► Text to Columns (or TEXTSPLIT in newer Excel 365).                                                                 | Scratch!B10<br><code>=SPLIT(Stores!E2," ")</code>                                                 |
| 7 | <b>Ranges With No Bottom</b><br>★★☆ · 20 XP           | In Scratch!B12, add up all Qty on the Sales sheet using an open-ended range that starts at row 2 and has no last row.<br><br>Takeaway: Open ranges grow automatically as you add rows. In Excel you would use E:E or a Table.                                              | Scratch!B12<br><code>=SUM(Sales!E2:E)</code>                                                      |
| 8 | <b>The CONCAT Gotcha</b><br>★★☆ · 15 XP               | In Scratch!B13, build Cumming, GA from Stores!B2 and Stores!C2. (Try =CONCAT(Stores!B2," ",Stores!C2) first and read the error!)                                                                                                                                           | Scratch!B13<br><code>=CONCATENATE(Stores!B2," ",Stores!C2)</code>                                 |

Takeaway: The same function name can have different rules in different apps.

Also accepted: `=Stores!B2&"`,  
`"&Stores!C2`

## 9 Pattern Hunter

★★★ · 25 XP

In Scratch!B14, count how many product names contain a digit (0-9) using REGEXMATCH.

Takeaway: Regular expressions are a mini-language for patterns. Google Sheets has them built in.

Scratch!B14

```
=SUMPRODUCT( --  
REGEXMATCH(Products!B2:B25, "\d")  
)
```

## Discussion

1. Which is worse: a formula that errors, or one that silently gives the wrong answer? Why?
2. Name one thing Google Sheets can do that Excel cannot, and one the other way around.

## Exit ticket

In Google Sheets, sort Products!B2:D25 by price, lowest first.

Answer: `=SORT(Products!B2:D25, 3, TRUE)`

## Lesson 4: Data Files: CSV & TSV

Workspace: **CSV / TSV** · 8 challenges · about 45 minutes

CSV is how data moves between programs — and how most AI training data is shared. It is plain text: if you understand the rules, you can read and fix any dataset.

### Students will be able to

- Values vs formulas
- Quoting and escaping
- Delimiters
- How spreadsheets change data on import

### Common misconceptions

- Saving as CSV throws away formulas, formatting and other sheets.
- Opening a CSV in a spreadsheet can drop leading zeros and mangle long IDs.
- An extra comma in one row shifts every value after it.

### Vocabulary

|                       |                                                                                        |                            |
|-----------------------|----------------------------------------------------------------------------------------|----------------------------|
| <b>Delimiter</b>      | The character between values: comma (CSV) or tab (TSV).                                | SKU-101,Granola Bar,Snacks |
| <b>Field / record</b> | A field is one value; a record is one line (row).                                      | 4 fields: a,b,c,d          |
| <b>Quoting</b>        | Values containing the delimiter, quotes or line breaks are wrapped in "double quotes". | "Gel Pens, 4-pack"         |
| <b>Escaping</b>       | A quote inside a quoted value is written twice.                                        | "12"" Ruler"               |
| <b>Type inference</b> | When a program opens a CSV it GUESSES types — and can change your data.                | 05401 → 5401               |

### Challenges & answer key

| # | Challenge                                      | Task                                                                                                                                                                                                                                                           | Answer                |
|---|------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------|
| 1 | <b>Where Did My Formula Go?</b><br>☆☆☆ · 15 XP | You save an Excel workbook as CSV. Cell B2 had =SUM(D2:D25) showing \$189.63. What is stored in the CSV file?<br><br>Takeaway: CSV keeps the RESULT, not the formula. Re-open it and your formulas are gone. Try "Export from a spreadsheet" to see it happen. | <b>Answer:</b> 189.63 |

|   |                                             |                                                                                                                                                                                                                                                                                      |                                                                                            |
|---|---------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| 2 | <b>Export a Sheet</b><br>★☆☆ · 15 XP        | Click Export from spreadsheet, choose the Excel 365 → Products sheet and export it as CSV. Look at the "What was lost" report.<br><br>Takeaway: Any formulas you added on that sheet became plain numbers in the CSV.                                                                | Export from spreadsheet ► Excel 365 ► Products ► CSV.                                      |
| 3 | <b>Fix the Broken Comma</b><br>★☆☆ · 25 XP  | Open broken_products.csv. One row has too many fields because a product name contains a comma. Fix it so every row has 8 fields.<br><br>Takeaway: Quotes protect commas inside values. This is why TSV (tabs) is popular — names rarely contain tabs.                                | In broken_products.csv change Gel Pens, 4-pack to "Gel Pens, 4-pack"                       |
| 4 | <b>Commas to Tabs</b><br>★☆☆ · 20 XP        | Create a new file named stores.tsv that holds the same data as stores.csv, but separated by TAB characters.<br><br>Takeaway: Same data, different delimiter. Spreadsheets can open both.                                                                                             | New file stores.tsv, paste stores.csv, Find & replace , → \t (Replace all).                |
| 5 | <b>The Vanishing Zero</b><br>★☆☆ · 15 XP    | Open suppliers.csv and switch the preview to "Open like a spreadsheet would". What happens to Lakeside Apparel's ZIP code 05401?<br><br>Takeaway: ZIP codes, phone numbers and IDs are TEXT, not numbers. Import them as text or the zeros disappear forever.                        | <b>Answer:</b> It becomes 5401                                                             |
| 6 | <b>Quotes Inside Quotes</b><br>★★★ · 25 XP  | Add a new product to the end of products.csv: SKU SKU-307, product 12" Ruler (that's 12 inches!), category School Supplies, price 1.99, cost 0.6, InStock 30, ReorderAt 10, supplier Office Hub Supply.<br><br>Takeaway: CSV escaping rule: wrap in quotes, double any inner quotes. | Add to products.csv: SKU-307,"12"" Ruler",School Supplies,1.99,0.6,30,10,Office Hub Supply |
| 7 | <b>Count the Fields</b><br>★☆☆ · 10 XP      | How many fields (values) are in this CSV line?SKU-302,"Gel Pens, 4-pack",School Supplies,4.5<br><br>Takeaway: SKU-302   Gel Pens, 4-pack   School Supplies   4.5 → 4 fields.                                                                                                         | <b>Answer:</b> 4                                                                           |
| 8 | <b>Pick the Right Format</b><br>★☆☆ · 10 XP | Which file type keeps your formulas, formatting and multiple sheets?<br><br>Takeaway: Use .xlsx (or a Google Sheet) for working files. Use CSV/TSV to move data between programs.                                                                                                    | <b>Answer:</b> .xlsx                                                                       |

## Discussion

1. You email a coworker a CSV of your gradebook. What did you lose?
2. Why would anyone choose TSV over CSV?

## Exit ticket

Write the CSV line for: Smith, Jo | 05401 | said "hi"

Answer: `"Smith, Jo",05401,"said ""hi"""`

# Lesson 5: Data Wrangling I: Cleaning Data for AI

Workspace: **Data Wrangling Lab** · 11 challenges · about 90 minutes

Data scientists spend most of their time cleaning data. An AI model learns whatever is in its training data — including typos, duplicates and fake zeros. Clean data is the difference between a useful model and a confident wrong one.

## Students will be able to

- Leading zeros and text vs numbers
- Finding blank and fake-null values
- Converting text to numbers
- Fixing mixed date formats
- TRIM, PROPER, UPPER, nested text formulas

## Common misconceptions

- Replacing missing values with 0 changes averages — missing is not zero.
- 03.09.2026 is September 3 in Europe and March 9 in the US.
- Clean into NEW columns so you can always compare with the original.

## Vocabulary

|                         |                                                                      |                                 |
|-------------------------|----------------------------------------------------------------------|---------------------------------|
| Data type               | Number, text, date, TRUE/FALSE. Digits can be TEXT ("0104").         | =ISTEXT(C2)                     |
| Null / missing value    | No value. Can hide as blanks or as words like N/A, -, null.          | =COUNTBLANK(E2:E31)             |
| Type conversion         | Changing a value's type.                                             | =VALUE("12") =TEXT(104, "0000") |
| Normalization           | Making the same thing look the same: case, spaces, formats.          | =UPPER(TRIM(B2))                |
| Date serial             | Real dates are numbers (days since 1900), so they sort and subtract. | =DATEVALUE("Sep 4, 2026")       |
| Garbage in, garbage out | A model can only be as good as its data.                             | Open the 🤖 AI check tab         |

## Challenges & answer key

| # | Challenge | Task | Answer |
|---|-----------|------|--------|
|---|-----------|------|--------|

|   |                                                      |                                                                                                                                                                                                                                                                                                                |                                                                                                                                |
|---|------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| 1 | <b>Where Did the Zero Go?</b><br>★★★ · 10 XP         | <p>RawOrders!C2 shows 104, but the real item code is 0104. Why?</p> <p>Takeaway: IDs, ZIP codes and phone numbers are TEXT, even though they are made of digits. An AI model will treat 104 and "0104" as two different items.</p>                                                                             | <p><b>Answer:</b> It was stored as a NUMBER, and numbers never keep leading zeros</p>                                          |
| 2 | <b>Rebuild the Item Codes</b><br>★★★ · 20 XP         | <p>Type CleanCode in RawOrders!I1. In I2:I31, turn every ItemCode (column C) into a 4-character text code like 0104 — whether it arrived as 104, "0104" or "0402".</p> <p>Takeaway: TEXT(x, "0000") is the spreadsheet version of zero-padding. SQL: printf('%04d', x). Python: str(x).zfill(4).</p>           | <p>RawOrders!I1</p> <p>CleanCode</p> <p>RawOrders!I2:I31 (type in the first cell, fill down)</p> <p>=TEXT(C2, "0000")</p>      |
| 3 | <b>Count the Holes</b><br>★★★ · 10 XP                | <p>In Report!B2, count how many Qty cells (RawOrders!E2:E31) are completely empty.</p> <p>Takeaway: Empty is only ONE kind of missing data. Look at the next challenge...</p>                                                                                                                                  | <p>Report!B2</p> <p>=COUNTBLANK(RawOrders!E2:E31)</p>                                                                          |
| 4 | <b>Fake Nulls</b><br>★★★ · 20 XP                     | <p>Some systems write missing values as words. In Report!B3, count Qty cells that say N/A, - or null.</p> <p>Takeaway: COUNTBLANK said 2 — the real number of missing quantities is 5. "null", "N/A" and "-" are TEXT that MEANS missing. Before training a model, they must all become one true blank/NA.</p> | <p>Report!B3</p> <p>=COUNTIF(RawOrders!E2:E31, "N/A")+COUNTIF(RawOrders!E2:E31, "-")+COUNTIF(RawOrders!E2:E31, "null")</p>     |
| 5 | <b>One Kind of Missing</b><br>★★★ · 25 XP            | <p>Type CleanQty in J1. In J2:J31, turn Qty into a real number — "12" (text) becomes 12 — and turn every kind of missing value into a blank "".</p> <p>Takeaway: Text that looks like a number does not add up: SUM ignores "12". Converting types is step one of every data-cleaning job.</p>                 | <p>RawOrders!J1</p> <p>CleanQty</p> <p>RawOrders!J2:J31 (type in the first cell, fill down)</p> <p>=IFERROR(VALUE(E2), "")</p> |
| 6 | <b>Same Store, Different Spelling</b><br>★★★ · 15 XP | <p>Type CleanStore in K1. In K2:K31, clean the Store codes so "s04" and " S02 " become S04 and S02.</p> <p>Takeaway: To a computer "S04", "s04" and " S04" are three different stores. A model trained on this would think you have 13 stores!</p>                                                             | <p>RawOrders!K1</p> <p>CleanStore</p> <p>RawOrders!K2:K31 (type in the first cell, fill down)</p> <p>=UPPER(TRIM(B2))</p>      |
| 7 | <b>Prices That Are Really Text</b><br>★★★ · 15 XP    | <p>Type CleanPrice in L1. In L2:L31, turn UnitPrice into real numbers ("\$.3.25" → 3.25, "14.99 " → 14.99).</p>                                                                                                                                                                                                | <p>RawOrders!L1</p> <p>CleanPrice</p>                                                                                          |

|                                                        |                                                                                                                                                                                                                                                                                                                                                                     |                                                                                                                                                                                                                                            |
|--------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                        | <p>Takeaway: If a column has even ONE text price, averages and sums silently skip it. Always check types.</p>                                                                                                                                                                                                                                                       | <p>RawOrders!L2:L31 (type in the first cell, fill down)</p> <p>=VALUE(F2)</p>                                                                                                                                                              |
| <p>8 <b>Four Kinds of Dates</b></p> <p>★★★ · 30 XP</p> | <p>Type CleanDate in M1. In M2:M31, make every OrderDate a real date. Some are already dates, others are text like "2026-09-12", "9/23/2026" or "Sep 4, 2026". (Row 19 is special — skip it for now.)</p> <p>Takeaway: Real dates are numbers (days since 1900), so you can sort them, subtract them and group them by month. Text dates cannot do any of that.</p> | <p>RawOrders!M1</p> <p>CleanDate</p> <p>RawOrders!M2:M31 (type in the first cell, fill down)</p> <p>=IF( ISNUMBER(D2) ,D2 ,DATEVALUE(D2) )</p> <p>Row 19 (European date) shows #VALUE! here — that is the next challenge.</p>              |
| <p>9 <b>The European Date</b></p> <p>★★★ · 30 XP</p>   | <p>RawOrders!D19 says 03.09.2026 — in Europe that means 3 September 2026. DATEVALUE cannot read it. In Report!B5, build the real date from its pieces.</p> <p>Takeaway: Is 03/09 March 9th or September 3rd? It depends on the country. Merging data from different places is one of the most common ways datasets go wrong.</p>                                    | <p>Report!B5</p> <p>=DATE(RIGHT(RawOrders!D19,4) ,MID(RawOrders!D19,4,2) ,LEFT(RawOrders!D19,2) )</p>                                                                                                                                      |
| <p>10 <b>Fix the Names</b></p> <p>★★★ · 35 XP</p>      | <p>Type CleanName in N1. In N2:N31, turn every Customer into First Last with proper capitals: "jordan smith", " Avery Johnson ", "TAYLOR KIM" and even "Davis, Morgan" → "Morgan Davis".</p> <p>Takeaway: This is a nested formula: IF, ISNUMBER, FIND, PROPER, TRIM, MID and LEFT working together. Building it one piece at a time is the skill.</p>              | <p>RawOrders!N1</p> <p>CleanName</p> <p>RawOrders!N2:N31 (type in the first cell, fill down)</p> <p>=IF( ISNUMBER(FIND(" ",G2) ) ,PROPER(TRIM(MID(G2,FIND(" ",G2)+1,99) ) )&amp;" "&amp;LEFT(G2,FIND(" ",G2) - 1) ,PROPER(TRIM(G2) ) )</p> |
| <p>11 <b>How Dirty Was It?</b></p> <p>★★★ · 10 XP</p>  | <p>After CleanQty is done, count in Report!B4 how many orders have no usable quantity.</p> <p>Takeaway: Report your missing data. A model trained without telling anyone that 1 in 6 rows was dropped can be badly biased.</p>                                                                                                                                      | <p>Report!B4</p> <p>=COUNTBLANK(RawOrders!J2:J31)</p> <p>Needs earlier step(s): wr-qty</p>                                                                                                                                                 |

## Discussion

1. An AI model was trained on data where missing quantities were replaced with 0. What will it get wrong?
2. Is 03/04/2026 March 4 or April 3? How would you find out for a real dataset?
3. Why clean into a NEW column instead of overwriting the original?

### Exit ticket

A column of ZIP codes shows 2110 and 5401. Write the formula that fixes them.

Answer: `=TEXT(A2, "00000")`

# Lesson 6: Data Wrangling II: Combining & Lookups

Workspace: **Data Wrangling Lab** · 11 challenges · about 90 minutes

Real datasets live in separate tables. Joining them — matching orders to products, stores to regions — only works when the keys match exactly. This is the same idea as a JOIN in SQL or merge() in Python.

## Students will be able to

- VLOOKUP and HLOOKUP
- Why lookups fail (type mismatch)
- Concatenation and delimiters
- TEXTJOIN
- Filtering around missing values

## Common misconceptions

- Clean the key column BEFORE looking up.
- Lock lookup ranges with \$ before filling down.
- Blank text "" counts as bigger than any number in comparisons.

## Vocabulary

|                          |                                                                |                                               |
|--------------------------|----------------------------------------------------------------|-----------------------------------------------|
| <b>Key</b>               | The column used to match rows between tables.                  | Item code 0104                                |
| <b>VLOOKUP / HLOOKUP</b> | Look up down the first column / across the first row.          | =HLOOKUP("S03", Targets!A1:F3, 3, FALSE)      |
| <b>Type mismatch</b>     | Number 104 never equals text "0104" — the lookup returns #N/A. | =VLOOKUP(C2, ...) → #N/A                      |
| <b>Concatenation</b>     | Joining text together.                                         | =B2 & ", " & A2                               |
| <b>Delimited list</b>    | Several values in one cell separated by a character.           | snacks drinks apparel                         |
| <b>Filtering</b>         | Keeping only rows that pass a test — watch out for blanks.     | =FILTER(A2:A31, ISNUMBER(J2:J31)*(J2:J31>10)) |

## Challenges & answer key

| # | Challenge                              | Task                                                                                                                 | Answer                                                                                   |
|---|----------------------------------------|----------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| 1 | <b>The #N/A Mystery</b><br>★☆☆ · 15 XP | =VLOOKUP(RawOrders!C2, ItemCodes!A2:D25, 2, FALSE) gives #N/A, even though code 0104 IS in the ItemCodes table. Why? | <b>Answer:</b> C2 is the NUMBER 104 and the table has the TEXT "0104" — they never match |

Takeaway: A lookup only matches the same TYPE: number 104 ≠ text "0104". This is the #1 reason lookups fail on real data.

|   |                                              |                                                                                                                                                                                                                                                                            |                                                                                                                                                                                                                                       |
|---|----------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 2 | <b>Look Up the Products</b><br>★★★ · 20 XP   | Type Product in O1. In O2:O31, use VLOOKUP with your CleanCode (column I) to show each product name from ItemCodes.<br><br>Takeaway: Clean first, then join. In SQL this is a JOIN; in Python pandas it is merge().                                                        | RawOrders!O1<br><b>Product</b><br><br>RawOrders!O2:O31 (type in the first cell, fill down)<br><b>=VLOOKUP(I2,ItemCodes!\$A\$2:\$D\$25,2,FALSE)</b><br><br>Needs earlier step(s): wr-code                                              |
| 3 | <b>Line Totals</b><br>★★★ · 25 XP            | Type LineTotal in P1. In P2:P31, calculate CleanQty × the ItemCodes price. If CleanQty is blank, leave LineTotal blank.<br><br>Takeaway: Missing values spread: blank × price should stay blank, not turn into 0 — a fake 0 would teach a model that people order nothing. | RawOrders!P1<br><b>LineTotal</b><br><br>RawOrders!P2:P31 (type in the first cell, fill down)<br><b>=IF(J2="", "", J2*XL00KUP(I2,ItemCodes!\$A\$2:\$A\$25,ItemCodes!\$D\$2:\$D\$25))</b><br><br>Needs earlier step(s): wr-code, wr-qty |
| 4 | <b>HLOOKUP Goes Sideways</b><br>★★★ · 15 XP  | The Targets sheet is laid out sideways (stores across the top). In Report!B7, use HLOOKUP to find the Sept Target for store S03.<br><br>Takeaway: VLOOKUP = vertical tables, HLOOKUP = horizontal tables. XLOOKUP and INDEX/MATCH handle both.                             | Report!B7<br><b>=HLOOKUP("S03",Targets!A1:F3,3,FALSE)</b>                                                                                                                                                                             |
| 5 | <b>Region for Every Order</b><br>★★★ · 20 XP | Type Region in Q1. In Q2:Q31, use HLOOKUP with your CleanStore (column K) to show each order's region.<br><br>Takeaway: Every lookup in this lab worked only because you cleaned the key column first.                                                                     | RawOrders!Q1<br><b>Region</b><br><br>RawOrders!Q2:Q31 (type in the first cell, fill down)<br><b>=HLOOKUP(K2,Targets!\$B\$1:\$F\$2,2,FALSE)</b><br><br>Needs earlier step(s): wr-store                                                 |
| 6 | <b>Now the Math Works</b><br>★★★ · 10 XP     | In Report!B8, add up all the cleaned quantities (CleanQty).<br><br>Takeaway: SUM(E2:E31) gives a smaller, WRONG total because it skips text numbers. Clean data changes the answer.                                                                                        | Report!B8<br><b>=SUM(RawOrders!J2:J31)</b><br><br>Needs earlier step(s): wr-qty                                                                                                                                                       |
| 7 | <b>Last, First</b><br>★★★ · 15 XP            | On Contacts, fill G2:G9 with names written as Last, First (e.g. "Whitfield, Dana").<br><br>Takeaway: A delimiter is the character that separates pieces of text — here it is ", ".                                                                                         | Contacts!G2:G9 (type in the first cell, fill down)<br><b>=B2&amp;" , "&amp;A2</b>                                                                                                                                                     |
| 8 | <b>Keep the Zero (Again)</b><br>★★★ · 25 XP  | In Contacts!H2:H9, build a mailing line like Burlington, VT 05401. Watch out:                                                                                                                                                                                              | Contacts!H2:H9 (type in the first cell, fill down)                                                                                                                                                                                    |

|                                                             |                                                                                                                                                                                                                                                                                                  |                                                                                                                                                 |
|-------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                             | <p>the Zip column lost its leading zeros!</p> <p>Takeaway: ZIP codes in New England start with 0. Treat ZIPs as text from the start and this problem never happens.</p>                                                                                                                          | <pre>=C2&amp;" , "&amp;D2&amp;" "&amp;TEXT(E2,"00000")</pre>                                                                                    |
| <p>9 <b>Count the Delimited Tags</b></p> <p>★★★ · 30 XP</p> | <p>Column F holds tags separated by a pipe: snacks drinks. In Contacts!I2:I9, count how many tags each contact has (blank = 0).</p> <p>Takeaway: Sheets has SPLIT(F2, " ") and Excel 365 has TEXTSPLIT to break the tags into cells. Counting delimiters is the trick that works everywhere.</p> | <p>Contacts!I2:I9 (type in the first cell, fill down)</p> <pre>=IF(F2="",0,LEN(F2)- LEN(SUBSTITUTE(F2," ",""))+1)</pre>                         |
| <p>10 <b>Write a CSV Line</b></p> <p>★★★ · 15 XP</p>        | <p>In Report!B10, join Contacts!A2:E2 into one CSV line with commas between the values.</p> <p>Takeaway: This is exactly what "Save as CSV" does for every row. Now you know why a comma INSIDE a value breaks the file.</p>                                                                     | <p>Report!B10</p> <pre>=TEXTJOIN(" ",FALSE,Contacts!A2:E2)</pre>                                                                                |
| <p>11 <b>Filter the Feed</b></p> <p>★★★ · 30 XP</p>         | <p>In Report!D2, use FILTER to list the OrderIDs whose CleanQty is more than 10. (Careful — try J2:J31&gt;10 alone first and look at what sneaks in!)</p> <p>Takeaway: Missing values break filters in sneaky ways. Always ask "what happens to the blanks?"</p>                                 | <p>Report!D2</p> <pre>=FILTER(RawOrders!A2:A31,ISNUMBER(RawOrders!J2:J31)* (RawOrders!J2:J31&gt;10))</pre> <p>Needs earlier step(s): wr-qty</p> |

## Discussion

1. VLOOKUP says #N/A but you can SEE the value in the table. List three reasons this happens.
2. Why do joins (lookups) come AFTER cleaning, never before?

## Exit ticket

Count the items in "red|green|blue" with one formula that works in every app.

Answer: 

```
=LEN(A2)-LEN(SUBSTITUTE(A2,"|",""))+1
```

# Lesson 7: SQL for Data Analysts

Workspace: **SQL Database** · 9 challenges · about 75 minutes

Companies keep their data in databases, and SQL is how you ask them questions. Every spreadsheet skill has a SQL twin — and SQL scales to millions of rows.

## Students will be able to

- SELECT, WHERE, ORDER BY
- GROUP BY and HAVING
- JOIN (the SQL VLOOKUP)
- Integer division
- UPDATE safely

## Common misconceptions

- Text uses 'single quotes'.
- Whole-number division drops decimals:  $7 / 2 = 3$ .
- UPDATE or DELETE without WHERE changes every row.

## Vocabulary

|                            |                                                            |                                                                          |
|----------------------------|------------------------------------------------------------|--------------------------------------------------------------------------|
| <b>SELECT ...<br/>FROM</b> | Choose columns from a table.                               | <code>SELECT product, price FROM products;</code>                        |
| <b>WHERE</b>               | Keep only matching rows (like FILTER).                     | <code>WHERE category = 'Snacks'</code>                                   |
| <b>GROUP BY</b>            | One result per group (like SUMIF for every value at once). | <code>SELECT store_id, SUM(qty) FROM sales<br/>GROUP BY store_id;</code> |
| <b>JOIN</b>                | Combine tables on a key (like VLOOKUP for every row).      | <code>FROM sales s JOIN products p ON s.sku =<br/>p.sku</code>           |
| <b>NULL</b>                | SQL's missing value. Test with IS NULL, never = NULL.      | <code>WHERE qty IS NULL</code>                                           |

## Challenges & answer key

| # | Challenge                               | Task                                                                                                                                                       | Answer                               |
|---|-----------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------|
| 1 | <b>SELECT Everything</b><br>☆☆☆ · 10 XP | Show every column and every row of the products table.<br><br>Takeaway: A spreadsheet shows all data all the time. A database only shows what you ask for. | <code>SELECT * FROM products;</code> |

|   |                                                     |                                                                                                                                                                                                                                                      |                                                                                                                                                                                             |
|---|-----------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 2 | <b>WHERE = FILTER</b><br>★★★ · 15 XP                | Show the product and price of every product in the Electronics category.<br><br>Takeaway: WHERE works like FILTER. Remember: 'single quotes' for text, and it is case-sensitive.                                                                     | <pre>SELECT product, price FROM products WHERE category = 'Electronics';</pre>                                                                                                              |
| 3 | <b>Top 3 Most Expensive</b><br>★★★ · 20 XP          | Show the product and price of the 3 most expensive products, highest first.<br><br>Takeaway: ORDER BY = SORT. LIMIT = "only the first N".                                                                                                            | <pre>SELECT product, price FROM products ORDER BY price DESC LIMIT 3;</pre>                                                                                                                 |
| 4 | <b>GROUP BY = SUMIF for Everyone</b><br>★★★ · 25 XP | Show each store_id with the total qty it sold (from the sales table).<br><br>Takeaway: One GROUP BY query does what five SUMIF formulas would.                                                                                                       | <pre>SELECT store_id, SUM(qty) FROM sales GROUP BY store_id;</pre>                                                                                                                          |
| 5 | <b>JOIN = VLOOKUP</b><br>★★★ · 30 XP                | For every sale, show the order_id and the product name. The name lives in the products table, so you need to JOIN.<br><br>Takeaway: A JOIN does a lookup for EVERY row at once — no filling down.                                                    | <pre>SELECT s.order_id, p.product FROM sales s JOIN products p ON s.sku = p.sku;</pre>                                                                                                      |
| 6 | <b>HAVING: Filter the Groups</b><br>★★★ · 30 XP     | Show the categories that have more than 5 products, with their count.<br><br>Takeaway: WHERE → rows. HAVING → groups.                                                                                                                                | <pre>SELECT category, COUNT(*) FROM products GROUP BY category HAVING COUNT(*) &gt; 5;</pre>                                                                                                |
| 7 | <b>Revenue by City</b><br>★★★ · 40 XP               | Show each store's city and its total revenue (qty × price), highest first. You need sales, stores AND products.<br><br>Takeaway: This would take lookups, a helper column and a pivot table in a spreadsheet. In SQL it is one query.                | <pre>SELECT st.city, SUM(s.qty * p.price) AS revenue FROM sales s JOIN stores st ON s.store_id = st.store_id JOIN products p ON s.sku = p.sku GROUP BY st.city ORDER BY revenue DESC;</pre> |
| 8 | <b>The Integer Division Trap</b><br>★★★ · 20 XP     | Find the average qty per sale with its decimals (it is NOT a whole number). Try SELECT SUM(qty) / COUNT(*) FROM sales; first and see what goes wrong.<br><br>Takeaway: In SQL, 7 / 2 = 3. In a spreadsheet, 7 / 2 = 3.5. Same math, different rules! | <pre>SELECT AVG(qty) FROM sales;</pre><br>SUM(qty)/COUNT(*) gives 4 (integer division) — the trap.                                                                                          |

---

## 9 Restock Day

★★★ · 25 XP

The truck arrived! For every product where `in_stock` is below `reorder_at`, set `in_stock` to `reorder_at × 2`. Leave the others alone. (If you mess up, use Reset database.)

```
UPDATE products SET in_stock =  
reorder_at * 2 WHERE in_stock <  
reorder_at;
```

Takeaway: Forget the WHERE and EVERY product changes. Real databases have no Ctrl+Z!

---

## Discussion

1. Which spreadsheet function is most like GROUP BY? Like JOIN? Like WHERE?
2. Why do real databases not have an Undo button — and what do companies do instead?

## Exit ticket

Write SQL for the number of sales per store.

Answer: 

```
SELECT store_id, COUNT(*) FROM sales GROUP BY store_id;
```

# Lesson 8: Cross-Platform Translator

Workspace: **Compare** · 5 challenges · about 25 minutes

The same formula can work, fail, or silently give a different answer depending on the app. Checking before you share is a professional habit.

## Students will be able to

- Which functions exist where
- Silent differences between apps
- Spreadsheet ideas in SQL

## Common misconceptions

- Test your file in the app your audience will use.
- Prefer functions that exist everywhere when sharing widely.

## Vocabulary

|                          |                                                             |                                                 |
|--------------------------|-------------------------------------------------------------|-------------------------------------------------|
| <b>Compatibility</b>     | Whether a formula works in another app or version.          | <code>XLOOKUP: Excel 365 ✓, Excel 2013 ✗</code> |
| <b>Silent difference</b> | No error, but a different result — the most dangerous kind. | <code>SORT(..., -1)</code> in Sheets            |
| <b>Translation</b>       | Rewriting a formula so it works elsewhere.                  | <code>XLOOKUP → INDEX/MATCH</code>              |

## Challenges & answer key

| # | Challenge                                 | Task                                                                                                                                                                             | Answer                                 |
|---|-------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------|
| 1 | <b>Who Says #NAME?</b><br>★☆☆ · 10 XP     | Where does <code>=XLOOKUP(A2, B:B, C:C)</code> give a <code>#NAME?</code> error?<br><i>Takeaway: Excel 2013 never got XLOOKUP. INDEX/MATCH works everywhere.</i>                 | <b>Answer:</b> Excel 2013              |
| 2 | <b>The Silent Sort</b><br>★★★ · 15 XP     | You type <code>=SORT(A2:C20, 3, -1)</code> in Google Sheets. What happens?<br><i>Takeaway: No error, wrong answer. Always check your results!</i>                                | <b>Answer:</b> It sorts low → high     |
| 3 | <b>Pointing at Nothing</b><br>★☆☆ · 10 XP | Cell A1 is empty. What does <code>=A1</code> show in Excel vs Google Sheets?<br><i>Takeaway: Little differences like this break checks such as <code>=IF(B2="", ...)</code>.</i> | <b>Answer:</b> Excel: 0, Sheets: blank |

---

4 **Array Formulas  
Through Time**

★ ★ ☆ · 15 XP

Which one needs Ctrl+Shift+Enter (or  
ARRAYFORMULA) to make  
=MAX(IF(C2:C25="Snacks", D2:D25))  
give the right answer?

**Answer:** Excel 2013 and Google  
Sheets

Takeaway: Excel 2013 needs  
Ctrl+Shift+Enter; Google Sheets needs  
ARRAYFORMULA (Ctrl+Shift+Enter adds it  
for you).

---

5 **Spreadsheet → SQL**

★ ★ ☆ · 10 XP

Which SQL clause does the same job as  
VLOOKUP?

**Answer:** JOIN

Takeaway: JOIN matches rows from two  
tables, like a lookup on every row at once.

---

## Discussion

1. Before sharing a spreadsheet, what should you check?

## Exit ticket

Name a function that exists in Excel 365 but gives #NAME? in Excel 2013.

Answer: XLOOKUP, FILTER, SORT, UNIQUE, IFS, TEXTJOIN, LET, MAXIFS...

# Lesson 9: Databases 101: What is an RDBMS?

Workspace: **RDBMS Explorer** · 10 challenges · about 60 minutes

Spreadsheets hold data; a relational database guards it. Real companies — and the AI systems they build — keep their important data in an RDBMS because it refuses bad data, connects tables with keys, and never leaves a change half-done.

## Students will be able to

- Tables, rows, columns and data types
- Primary keys and foreign keys
- Relationships and JOINS
- Constraints that refuse bad data
- Transactions and ACID

## Common misconceptions

- A spreadsheet that repeats the same fact in many rows will eventually disagree with itself.
- A foreign key must point at a row that already exists — add the parent first.
- Nothing inside BEGIN is permanent until COMMIT.

## Vocabulary

|                             |                                                                                            |                                   |
|-----------------------------|--------------------------------------------------------------------------------------------|-----------------------------------|
| <b>Table / row / column</b> | A table holds one kind of thing; each row is one item; each column is one typed attribute. | products: one row per product     |
| <b>Primary key</b>          | A column whose value is different for every row — the row's ID.                            | products.sku                      |
| <b>Foreign key</b>          | A column that holds another table's primary key, linking the two.                          | sales.sku → products.sku          |
| <b>Constraint</b>           | A rule the database enforces: NOT NULL, UNIQUE, CHECK, FOREIGN KEY, types.                 | CHECK (price >= 0)                |
| <b>Normalization</b>        | Store each fact once, in one place, and link to it.                                        | Price lives only in products      |
| <b>Transaction</b>          | Several changes that succeed or fail together.                                             | BEGIN; ... COMMIT; (or ROLLBACK;) |

## Challenges & answer key

| # | Challenge                            | Task                                                                                                                                     | Answer                    |
|---|--------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|---------------------------|
| 1 | <b>The R in RDBMS</b><br>☆☆☆ · 10 XP | What does the R in RDBMS stand for?<br><a href="#">Takeaway: Relational: data lives in tables, and keys create RELATIONSHIPS between</a> | <b>Answer:</b> Relational |

them.

|   |                                             |                                                                                                                                                                                                                                                                                                                         |                                                                                                                                                                                                                                                                                                                                                                   |
|---|---------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 2 | <b>Break the Spreadsheet</b><br>☆☆☆ · 15 XP | <p>In section 1. Grid vs. database, change the price in only ONE of the highlighted rows of the flat sheet. Watch the sheet start to disagree with itself.</p> <p>Takeaway: When the same fact is stored many times, copies drift apart. That is an "update anomaly". Databases avoid it by storing each fact once.</p> | Section 1: change only one of the highlighted Price cells in the flat sheet.                                                                                                                                                                                                                                                                                      |
| 3 | <b>Pick a Primary Key</b><br>☆☆☆ · 10 XP    | <p>You are building a students table for your school. Which column makes the best primary key?</p> <p>Takeaway: Names, grades and birthdays repeat. A student ID is unique — perfect for a primary key.</p>                                                                                                             | <b>Answer:</b> Student ID number                                                                                                                                                                                                                                                                                                                                  |
| 4 | <b>Follow the Key</b><br>☆☆☆ · 10 XP        | <p>sales.store_id holds values like S03 that must already exist in stores.store_id. What is sales.store_id?</p> <p>Takeaway: A foreign key points at another table's primary key. Following it is what a JOIN does.</p>                                                                                                 | <b>Answer:</b> A foreign key                                                                                                                                                                                                                                                                                                                                      |
| 5 | <b>The Database Says NO</b><br>☆☆☆ · 20 XP  | <p>Make the database refuse bad data in 3 different ways (duplicate key, missing value, broken rule, missing parent, wrong type...). Use section 4 or any console on this page.</p> <p>Takeaway: Constraints stop bad data at the door. In a spreadsheet you only find these problems later — while cleaning.</p>       | <pre>INSERT INTO products VALUES ('SKU-101', 'Fake', 'Snacks', 1, 0.4, 10, 5, 'X');</pre> <pre>INSERT INTO products (sku, category, price, in_stock) VALUES ('SKU-999', 'Snacks', 1, 5);</pre> <pre>UPDATE products SET price = -5 WHERE sku = 'SKU-101';</pre> <p>Any 3 of the 6 "Try it" buttons in section 4 (UNIQUE, NOT NULL, CHECK, FOREIGN KEY, type).</p> |
| 6 | <b>Build Your Own Table</b><br>☆☆☆ · 25 XP  | <p>In Free play, create a table students with student_id INTEGER PRIMARY KEY and name TEXT NOT NULL, then insert at least 2 students.</p> <p>Takeaway: An INTEGER PRIMARY KEY numbers new rows automatically — no duplicates, ever.</p>                                                                                 | <pre>CREATE TABLE students (student_id INTEGER PRIMARY KEY, name TEXT NOT NULL);</pre> <pre>INSERT INTO students (name) VALUES ('Ana'), ('Ben');</pre>                                                                                                                                                                                                            |
| 7 | <b>Connect Two Tables</b><br>☆☆☆ · 30 XP    | <p>Create enrollments with student_id INTEGER REFERENCES students(student_id) and course TEXT NOT NULL. Add an enrollment for one</p>                                                                                                                                                                                   | <pre>CREATE TABLE students (student_id INTEGER PRIMARY KEY, name TEXT NOT NULL);</pre>                                                                                                                                                                                                                                                                            |

|    |                                                                                                                                                                                                                                     |                                                                                                                                                                                                                                                                             |                                                                                                                                   |
|----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
|    | <p>of your students. (Then try enrolling student 99 and watch it fail!)</p> <p>Takeaway: You just built a relationship: one student, many enrollments — and the database guarantees every enrollment belongs to a real student.</p> | <pre>INSERT INTO students (name) VALUES ('Ana'), ('Ben');  CREATE TABLE enrollments (student_id INTEGER REFERENCES students(student_id), course TEXT NOT NULL);  INSERT INTO enrollments VALUES (1, 'Intro to AI');</pre> <p>Build students first (previous challenge).</p> |                                                                                                                                   |
| 8  | <p><b>Undo With ROLLBACK</b></p> <p>★★★ · 20 XP</p>                                                                                                                                                                                 | <p>In section 5, start a transaction, DELETE every sale, check they are gone — then ROLLBACK and bring all 60 back.</p> <p>Takeaway: Inside a transaction nothing is permanent until COMMIT. ROLLBACK is a real undo button for the whole database.</p>                     | <pre>BEGIN; DELETE FROM sales; SELECT COUNT(*) FROM sales; ROLLBACK;</pre>                                                        |
| 9  | <p><b>Survive the Power Failure</b></p> <p>★★★ · 25 XP</p>                                                                                                                                                                          | <p>In the lunch-money demo (section 5), start with a transaction, pull the plug with ⚡ Power failure!, then do the whole transfer again and COMMIT it.</p> <p>Takeaway: Atomicity: a transfer is either completely done or not done at all — money never vanishes.</p>      | <p>Section 5: With a transaction (BEGIN) → Step 1 → ⚡ Power failure! → With a transaction (BEGIN) → Step 1 → Step 2 → COMMIT.</p> |
| 10 | <p><b>All or Nothing</b></p> <p>★★★ · 10 XP</p>                                                                                                                                                                                     | <p>In a database transaction, atomic means...</p> <p>Takeaway: Atomic = indivisible. ACID: Atomic, Consistent, Isolated, Durable.</p>                                                                                                                                       | <p><b>Answer:</b> Either every change happens or none of them do</p>                                                              |

## Discussion

1. Your school keeps student records. Why would it use a database instead of one giant spreadsheet?
2. Give an example from your own life where "all or nothing" matters (like the lunch-money transfer).
3. Constraints stop bad data before it is saved. How would that have helped in the Data Wrangling Lab?

## Exit ticket

Write the CREATE TABLE for clubs(club\_id, name) where club\_id is the primary key and name is required.

Answer: 

```
CREATE TABLE clubs (club_id INTEGER PRIMARY KEY, name TEXT NOT NULL);
```

## Appendix: what is different in each tool

### Excel 365

- Dynamic arrays: one formula can **spill** results into the cells below it. If something is in the way you get `#SPILL!`.
- Has XLOOKUP, XMATCH, FILTER, SORT, UNIQUE, SEQUENCE, LET, IFS, SWITCH, TEXTJOIN, CONCAT, MAXIFS and MINIFS.
- Refer to a whole spilled result with `A2#` (the spill operator). Google Sheets does not have this.
- `SORT(range, 3, -1)` sorts column 3 high→low. Sort order is **1 or -1**.
- `FILTER(range, include, [if_empty])` takes ONE include test. Combine tests with `*` (AND) or `+` (OR).
- Open-ended ranges like `A2:A` are **not** allowed. Use `A:A` or `A2:A1000`.
- A formula pointing at an empty cell shows **0**.
- Excel refuses to accept a formula with a typo and shows a **"There's a problem with this formula"** pop-up.

### Excel 2013

- **No** XLOOKUP, FILTER, SORT, UNIQUE, LET, IFS, SWITCH, TEXTJOIN, CONCAT, MAXIFS, MINIFS → typing them gives `#NAME?`.
- Lookups: use `VLOOKUP(..., FALSE)` or the classic combo `INDEX(return_range, MATCH(value, lookup_range, 0))`.
- Instead of IFS, nest IFs: `IF(test1, a, IF(test2, b, c))`.
- No spilling. A formula that works on a whole range must be confirmed with **Ctrl+Shift+Enter**. Excel shows it as `{=MAX(IF(...))}`.
- Without Ctrl+Shift+Enter, a range used where one value is expected is cut down to the value in the **same row** (implicit intersection) — often the wrong answer!
- `SUMPRODUCT` handles arrays without Ctrl+Shift+Enter, which is why it was so popular.
- Join text with `CONCATENATE(a, b, c)` or `a & b & c`.

### Google Sheets

- `FILTER(range, condition1, condition2, ...)` — each condition is its **own argument**, and there is **no if\_empty**. No matches → `#N/A`.
- `SORT(range, column, is_ascending)` uses **TRUE/FALSE**. Typing Excel's `-1` still sorts ascending (any non-zero number counts as TRUE)!
- Operators on whole ranges (like `D2:D25*F2:F25`) need `ARRAYFORMULA( )`. Ctrl+Shift+Enter adds it for you.
- `CONCAT` only joins **two** values. Use `CONCATENATE`, `JOIN`, `TEXTJOIN` or `&` for more.
- Open-ended ranges like `A2:A` work (from row 2 to the bottom).
- Sheets-only functions: QUERY, ARRAYFORMULA, SPLIT, JOIN, REGEXMATCH, REGEXEXTRACT, REGEXREPLACE, COUNTUNIQUE, IMPORTRANGE, GOOGLEFINANCE.
- A bad formula is still saved — the cell shows `#ERROR!` (parse error) instead of a pop-up.
- A formula pointing at an empty cell shows a **blank**, not 0.

### CSV / TSV

- A CSV/TSV file stores **values only**. Saving as CSV turns every formula into its current result.
- No formatting (bold, colors, \$ signs as formats), and only **one sheet** per file.
- A value that contains a comma must be wrapped in "double quotes": "Gel Pens, 4-pack".
- A double quote inside a quoted value is written twice: "12"" Ruler".
- TSV uses the TAB character, so commas inside values are no problem.
- Spreadsheets "helpfully" convert values when they open a CSV: 05401 → 5401, long numbers → 4.41712E+15.

## SQL Database

- Text goes in '**single quotes**'. "Double quotes" mean a column name.
- Text comparisons are **case-sensitive**: 'snacks' does not equal 'Snacks'. LIKE is not case-sensitive.
- Dividing two whole numbers drops the decimals: 7 / 2 = 3. Use 7 / 2.0 or AVG().
- Columns are referred to by name, not letter: price, not D.
- Joining tables (JOIN) replaces VLOOKUP. Grouping (GROUP BY) replaces SUMIF/COUNTIF.
- UPDATE and DELETE without WHERE change **every row**. There is no undo in a real database!

## Appendix: error codes

|         |                                                                                             |
|---------|---------------------------------------------------------------------------------------------|
| #NAME?  | The formula contains unrecognized text (usually a misspelled or unsupported function name). |
| #VALUE! | Wrong type of value — e.g. doing math on text, or a range where one value was expected.     |
| #REF!   | The formula refers to a cell or sheet that does not exist.                                  |
| #DIV/0! | Dividing by zero (or by an empty cell).                                                     |
| #N/A    | A value is not available — usually a lookup that found no match.                            |
| #NUM!   | A number is invalid or too big/small.                                                       |
| #NULL!  | Ranges do not intersect.                                                                    |
| #SPILL! | The result is an array, but cells it needs to spill into are not empty.                     |
| #CALC!  | The calculation engine hit something it does not support, like an empty array.              |
| #ERROR! | Google Sheets could not understand (parse) the formula.                                     |